

GHID TEHNIC DE IMPLEMENTARE

Protectie anti-DDoS & anti-bruteforce pentru Drupal

nginx rate limiting · log processing custom · fail2ban ·
nftables

PUBLIC / SANITIZED

ERNU.EU — IT & Security Consulting

Contact: ITSec@enu.md

Data: 14 septembrie 2026 · Versiune document: 1.0

Cuprins

1. Arhitectura solutiei
2. Cerinte preliminare
3. Structura arhivei livrate
4. Instalare pas cu pas
5. Custom application log processing & integrarea cu fail2ban [sectiune detaliata](#)
6. Testare si validare
7. Monitorizare si mentenanta
8. Troubleshooting
9. Anexa: lista completa de fisiere

1. Arhitectura solutiei

Solutia acopera doua niveluri distincte de atac, fiecare cu instrumentul lui potrivit. Este esential de retinut ca **nginx si fail2ban actioneaza exclusiv la Layer 7**; un atac volumetric masiv (SYN flood la scara mare, UDP amplification) trebuie oprit in amonte, la nivel de provider/scrubbing, deoarece nicio configuratie locala nu poate absorbi un flood de zeci de Gbps.

1. **Retea (L3/L4)** — nftables: limiteaza rata de conexiuni noi, SYN flood, ICMP flood ↓
2. **nginx (L7)** — rate limiting per zona, timeouts anti-slowloris, deny pe cai sensibile ↓
3. **Log custom "security"** — nginx scrie DOAR evenimentele relevante (4xx/5xx, cereri suspecte) ↓
4. **security_log_processor.py** — coreleaza semnale slabe in scor de risc per IP ↓
5. **security-alerts.log** — linii normalizate `SECURITY_ALERT` ↓
6. **fail2ban** — citește alerta, aplica politica (maxretry/findtime/bantime) ↓
7. **nftables (set cu timeout)** — IP-ul e blocat la nivel de kernel, inainte sa ajunga la nginx

Pasii 3-6 formeaza **pipeline-ul de "custom application log processing"** — subiectul central al acestui ghid — si sunt tratati in detaliu in Capitolul 5.

2. Cerinte preliminare

Componenta	Versiune minima	Observatii
Drupal	10.x / 11.x	Testat pe structura standard <code>web/</code> ca document root
nginx	1.22+	Modulele <code>ngx_http_limit_req_module</code> si <code>ngx_http_limit_conn_module</code> (activate implicit in build-urile standard)
PHP-FPM	8.2 / 8.3	Socket unix recomandat fata de TCP (mai putin overhead)
fail2ban	0.11+	Pentru suport <code>bantime.increment</code> si actiuni multi-familie (IPv4/IPv6)
nftables sau iptables+ipset	0.9.3+ / oricare	Doua variante echivalente incluse — alegeti una (Capitolul 5.8)
Python	3.9+	Doar biblioteca standard — fara dependente externe
Acces	root / sudo	Pentru nginx, fail2ban, nftables, systemd

Atentie: daca site-ul e in spatele unui reverse-proxy, load-balancer sau CDN (Cloudflare etc), configurati OBLIGATORIU `nginx/conf.d/01-realip.conf` inainte de a activa rate limiting-ul. Altfel, toate limitele se vor aplica pe IP-ul proxy-ului, nu al vizitatorului real, si veti bloca tot traficul legitim.

3. Structura arhivei livrate

Arhiva `drupal-nginx-antiddos.zip` contine urmatoarele fisiere, gata de copiat pe server (caile tinta sunt indicate in `README.md` din arhiva):

```
archive/
├── nginx/
│   ├── conf.d/00-security-globals.conf    # zone rate-limit, timeouts, log_format custom
│   ├── conf.d/01-realip.conf              # config real IP pentru CDN/LB
│   └── sites-available/drupal-site.conf    # server block complet, hardened
├── fail2ban/
│   ├── filter.d/nginx-limit-req.conf
│   ├── filter.d/nginx-botsearch.conf
│   ├── filter.d/drupal-security-events.conf
│   ├── jail.d/drupal-nginx.local           # varianta NFTABLES
│   ├── jail.d/drupal-nginx-iptables.local  # varianta IPTABLES+ipset
│   ├── action.d/nftables-drupal-ban.conf   # varianta NFTABLES
│   └── action.d/iptables-drupal-ban.conf   # varianta IPTABLES+ipset
├── scripts/
│   ├── security_log_processor.py           # procesorul custom (Capitolul 5)
│   └── test_attack_simulation.sh           # simulare atac pentru testare pe staging
├── systemd/
│   └── drupal-security-processor.service
├── nftables/
│   └── 00-ddos-base.nft                   # protectie L3/L4 [varianta NFTABLES]
├── iptables/
│   └── 00-ddos-base-iptables.sh           # protectie L3/L4 [varianta IPTABLES]
└── README.md
```

Cele doua variante de firewall (nftables si iptables+ipset) sunt **echivalente functional** si complet interschimbabile — alegeti una singura, in functie de ce ruleaza deja pe server. Fisierele nginx, filtrele fail2ban si procesorul Python sunt identice pentru ambele.

4. Instalare pas cu pas

4.1 Nivelul retea — optiunea A: nftables (recomandat)

shell

```
mkdir -p /etc/nftables.d
cp nftables/00-ddos-base.nft /etc/nftables.d/
nft -f /etc/nftables.d/00-ddos-base.nft
echo 'include "/etc/nftables.d/*.nft"' >> /etc/nftables.conf
systemctl enable nftables
```

4.1bis Nivelul retea — optiunea B: iptables + ipset

Folositi aceasta varianta doar daca sistemul nu are nftables disponibil. Nu aplicati ambele optiuni pe acelasi server.

shell

```
apt install -y ipset iptables-persistent
bash iptables/00-ddos-base-iptables.sh
netfilter-persistent save
```

4.2 nginx

shell

```
cp nginx/conf.d/*.conf /etc/nginx/conf.d/
cp nginx/sites-available/drupal-site.conf /etc/nginx/sites-available/
ln -s /etc/nginx/sites-available/drupal-site.conf /etc/nginx/sites-enabled/

# editati server_name, root, caile certificatelor si socket-ul php-fpm
nano /etc/nginx/sites-available/drupal-site.conf

nginx -t && systemctl reload nginx
```

Daca site-ul ruleaza in spatele unui CDN/LB, decomentati si completati rangurile IP corecte in `01-realip.conf` inainte de reload.

4.3 Procesorul custom de log

shell

```
mkdir -p /opt/drupal-antiddos/scripts
cp scripts/security_log_processor.py /opt/drupal-antiddos/scripts/
chmod +x /opt/drupal-antiddos/scripts/security_log_processor.py

# fisierele de log trebuie sa existe si sa fie scriptibile de utilizatorul serviciului
touch /var/log/nginx/security-events.log /var/log/nginx/security-alerts.log
chown www-data:adm /var/log/nginx/security-*.log
```

```
cp systemd/drupal-security-processor.service /etc/systemd/system/  
systemctl daemon-reload  
systemctl enable --now drupal-security-processor  
systemctl status drupal-security-processor
```

4.4 fail2ban

shell – comun pentru ambele variante

```
cp fail2ban/filter.d/*.conf /etc/fail2ban/filter.d/
```

shell – DOAR daca folositi nftables

```
cp fail2ban/jail.d/drupal-nginx.local /etc/fail2ban/jail.d/  
cp fail2ban/action.d/nftables-drupal-ban.conf /etc/fail2ban/action.d/
```

shell – DOAR daca folositi iptables+ipset

```
cp fail2ban/jail.d/drupal-nginx-iptables.local /etc/fail2ban/jail.d/drupal-nginx.local  
cp fail2ban/action.d/iptables-drupal-ban.conf /etc/fail2ban/action.d/
```

shell

```
# in ambele cazuri: adaugati IP-ul vostru de administrare in sectiunea ignoreip!  
nano /etc/fail2ban/jail.d/drupal-nginx.local  
  
fail2ban-client reload  
fail2ban-client status
```

5. Custom application log processing & integrarea cu fail2ban

Aceasta este sectiunea centrala a ghidului: cum se creeaza, cum functioneaza si cum se intretine pipeline-ul de procesare custom a logurilor, de la cererea HTTP bruta pana la blocarea IP-ului la nivel de firewall.

5.1 De ce nu ajunge fail2ban "din cutie"

Fail2ban, folosit clasic, citeste un fisier de log linie cu linie si numara de cate ori un regex se potriveste pentru acelasi `<HOST>` intr-o fereastră de timp (`findtime`); la depasirea `maxretry`, declanseaza actiunea de ban. Acest model functioneaza bine pentru semnale simple si repetitive (ex: autentificare SSH esuata), dar are trei limitari importante pentru un site Drupal expus public:

Limitare fail2ban "nativ"	Consecinta practica
Fiecare regex evalueaza o singura linie, izolat	Nu poate corela "un 404 pe /wp-admin" + "User-Agent gol" + "apoi un POST pe /user/login" ca fiind acelasi atacator care sonda apoi ataca
Nu mentine un scor cumulativ intre tipuri diferite de evenimente	Un atacator "prudent" care ramane sub pragul fiecarui jail individual, dar combina mai multe tehnici, trece neobservat
Nu poate aplica logica de business	Nu stie sa ignore, de exemplu, cereri legitime catre JSON:API cu paginare agresiva dar autorizata

Solutia: introducem un **strat de corelare intermediar** — `security_log_processor.py` — care joaca rolul unui mini-IDS aplicativ, intre logul brut nginx si fail2ban.

5.2 Formatul de log custom generat de nginx

In loc sa cerem procesorului sa parseze logul de acces standard (zgomotos — contine si trafic 100% legitim), definim in `00-security-globals.conf` un `log_format security` dedicat, plus o directiva conditionala care scrie o linie DOAR cand cererea e potential relevanta:

nginx/conf.d/00-security-globals.conf

```
log_format security '$time_iso8601 client=$remote_addr method=$request_method '
                    'uri="$request_uri" status=$status bytes=$body_bytes_sent '
                    'reqtime=$request_time ua="$http_user_agent" '
                    'ref="$http_referer" bot=$bad_bot probe=$probe_path '
                    'xff="$http_x_forwarded_for"';

...
access_log /var/log/nginx/security-events.log security if=$log_security;
```

Variabila `$log_security` e calculata prin combinarea a doua blocuri `map`: unul care marcheaza orice status 4xx/5xx, altul care marcheaza cereri "suspecte" (User-Agent gol sau de scanner cunoscut, cai tipice de scanare precum `/wp-admin`, `.env`, `.git`, sau semnaturi simple de SQLi/XSS in URI).

Rezultatul: `security-events.log` contine doar 2-5% din volumul lui `access.log`, dar concentreaza aproape tot ce merita analizat.

5.3 Arhitectura procesorului (`security_log_processor.py`)

Procesorul ruleaza continuu ca serviciu systemd si face patru lucruri:

a) Urmărirea fisierului ("tail -F" propriu)

Functia `follow()` deschide fisierul, sare la sfarsit, si citeste linie cu linie intr-o bucla. La fiecare iteratie fara continut nou, verifica daca `inode`-ul fisierului de pe disc s-a schimbat (semn ca `logrotate` a rotit logul); daca da, redeschide fisierul nou creat, fara sa piarda evenimente si fara sa se blocheze pe un file handle "orfan".

b) Parsarea liniei

Un singur regex (`LOG_LINE_RE`) extrage toate campurile din formatul `security` definit mai sus, folosind named groups Python — ceea ce face codul robust la reordonarea campurilor si usor de extins.

c) Corelarea si scorul de risc

Pentru fiecare IP, clasa `Correlator` mentine o stare (`IPState`) cu trei cozi glisante (deque): evenimentele scorate din ultima `WINDOW_SECONDS` (implicit 60s), timestampurile tuturor cererilor (pentru detectia de rafala), si hit-urile pe `/user/login` (pentru bruteforce). Fiecare linie primeste 0 sau mai multe "semnale slabe", fiecare cu o pondere:

Semnal	Scor	Motiv
<code>empty_user_agent</code>	3	UA gol e neobisnuit la trafic real de browser
<code>known_bad_bot</code>	6	UA de scanner cunoscut (nikto, sqlmap, masscan...)
<code>path_probe</code>	4	Cale tipica de scanare (wp-admin, .env, .git, phpmyadmin)
<code>sqli_xss_signature</code>	8	Semnatura SQLi/XSS in URI (union, select, sleep(), <script)
<code>status_404_burst</code>	1	Slab individual, dar se acumuleaza rapid la scanare
<code>status_403_repeat</code>	5	403 repetat — probing de ACL sau bruteforce
<code>login_endpoint_hit</code>	2	Orice hit pe /user/login
<code>login_bruteforce</code>	10	≥5 hit-uri pe /user/login in fereastra de 60s
<code>high_request_rate</code>	7	≥20 cereri in 10 secunde de la acelasi IP

Scorurile din fereastra glisanta se insumeaza per IP. Cand suma atinge pragul `ALERT_THRESHOLD` (implicit 12) — de exemplu doar doua semnale medii, sau unul singur puternic plus contextul — si nu s-a mai trimis o alerta pentru acel IP in ultimele `COOLDOWN_SECONDS` (300s, pentru a evita spam-ul de alerte identice), procesorul genereaza o alerta.

De ce corelare si nu un singur prag simplu? Un atacator care doar scaneaza `/wp-admin` o data nu merita ban (fals-pozitiv posibil). Dar acelasi IP care in 30 de secunde loveste 3 cai de scanare, foloseste un User-Agent gol SI incearca login — desi niciun semnal individual nu ar trece pragul unui jail simplu — acumuleaza rapid $4+6+2=12+$ puncte si declanseaza corect alerta.

d) Emiterea alertei normalizate

Cand pragul e depasit, procesorul scrie o singura linie, intr-un format fix, usor de parsat printr-un regex simplu:

```
/var/log/nginx/security-alerts.log
```

```
2026-09-14T10:22:45+0300 SECURITY_ALERT ip=203.0.113.7 score=18
reason=known_bad_bot,login_bruteforce,path_probe path=/user/login count=6 window=60s
```

Acest format e proiectat special pentru fail2ban: contine `<HOST>` intr-o pozitie fixa, usor de capturat cu regex, si informatia de `reason / score` ramane in log pentru audit uman, fara sa complice parsarea automata.

5.4 Filtrul fail2ban explicat

```
fail2ban/filter.d/drupal-security-events.conf
```

```
[Definition]
failregex = ^\S+ SECURITY_ALERT ip=<HOST> score=\d+ reason=\S+.*$
ignoreregex =
datepattern = ^%Y-%m-%dT%H:%M:%S%z
```

`<HOST>` este un tag special, intern fail2ban, care se extinde automat intr-un regex ce recunoaste adrese IPv4, IPv6 si hostname-uri — nu trebuie scris manual. `datepattern` ii spune lui fail2ban exact cum sa citeasca timestamp-ul din inceputul liniei (identic cu formatul ISO 8601 pe care il scrie procesorul Python), astfel incat fereastra `findtime` sa fie calculata corect chiar daca fail2ban proceseaza liniile cu o mica intarziere.

5.5 Jail-ul: de ce maxretry=1

```
fail2ban/jail.d/drupal-nginx.local – [drupal-security-events]
```

```
maxretry = 1
findtime = 120
bantime = 24h
action = nftables-drupal-ban[name=drupal-critical]
```

Spre deosebire de jail-urile clasice (unde `maxretry` ridicat compenseaza faptul ca fiecare linie individuala e un semnal slab), aici `maxretry=1` este corect: procesorul Python **a facut deja** agregarea si scorul. O singura linie `SECURITY_ALERT` reprezinta deja un tipar confirmat de atac, nu o cerere izolata — deci fail2ban trebuie sa actioneze imediat la prima aparitie. `bantime=24h` reflecta increderea

mai mare in acest semnal fata de jail-urile de nivel 1 (rate-limit simplu, 1h) si nivel 2 (scanare de boti, 6h).

Politica `bantime.increment` din `[DEFAULT]` face ca un IP recidivist (banat, deblocat, apoi banat din nou) sa primeasca de fiecare data un ban de doua ori mai lung, pana la maxim o saptamana — util impotriva atacatorilor care rotesc IP-uri dintr-un pool restrans sau revin dupa expirarea banului initial.

5.6 Actiunea: cum ajunge un IP sa fie efectiv blocat

Cand jail-ul decide banarea, fail2ban executa `actionban` din `nftables-drupal-ban.conf`. Secventa completa, de la cererea malitioasa pana la blocare:

1. Cererea HTTP ajunge la nginx si genereaza o linie in `security-events.log`
2. `security_log_processor.py` (rulare continua, latentă sub 1s) actualizeaza scorul IP-ului
3. Scorul depaseste pragul → se scrie linia `SECURITY_ALERT` in `security-alerts.log`
4. fail2ban (poll interval implicit ~1s pe fisierele monitorizate) detecteaza linia noua si o potriveste cu filtrul
5. Jail-ul `drupal-security-events` declanseaza `actionban`
6. Se executa `nft add element inet f2b-drupal drupal-critical { <ip> timeout 86400s }`
7. Urmatorul pachet de la acel IP e comparat cu setul in lantul `input` (hook la prioritate -10) si **DROP**-uit direct in netfilter/kernel

Punctul critic la pasul 7: pachetul e aruncat **inainte** sa ajunga la nivelul TCP/HTTP al nginx-ului. Comparativ cu un `deny` sau `return 403` in nginx (care presupune ca nginx a acceptat deja conexiunea TCP, a facut handshake TLS si a parsat request-ul HTTP inainte sa refuze), blocarea la nivel de firewall consuma cu ordine de marime mai putine resurse per pachet respins — relevant tocmai in scenariul unui atac cu volum mare.

Se foloseste un **set nftables** cu `flags timeout` in loc de o regula noua per IP din doua motive: (1) cautarea intr-un set e o operatie de hash, $O(1)$, indiferent daca sunt 10 sau 100.000 de IP-uri banate — o regula per IP ar insemna parcurgere secventiala, $O(n)$; (2) expirarea e gestionata nativ de nftables (`timeout <bantime>s`), fail2ban nemaifiind nevoit sa "tina minte" sa stearga regula la momentul potrivit — desi `actionunban` ramane definit pentru cazul unui unban manual (`fail2ban-client set drupal-security-events unbanip <ip>`).

Persistenta la reboot: seturile nftables create dinamic de fail2ban (tabela `inet f2b-drupal`) NU supravietuiesc unui restart de server implicit. La pornire, fail2ban recreeaza automat tabela goala (`actionstart`) si repopuleaza banurile active din baza sa proprie de date (`/var/lib/fail2ban/fail2ban.sqlite3`), daca `dbpurgeage` nu a expirat intre timp — comportament standard, nu necesita configurare suplimentara.

5.8 Varianta iptables + ipset (alternativa la nftables)

Pentru servere unde nftables nu e disponibil (distributii mai vechi, sau politica interna care impune iptables clasic), pipeline-ul de mai sus ramane **identic pana la actiunea de banare** — nginx, `security_log_processor.py` si filtrele fail2ban nu se schimba deloc. Difera doar mecanismul prin care IP-ul ajunge efectiv blocat la nivel de kernel.

De ce ipset, si nu reguli iptables simple

Un iptables "clasic" ar necesita o regula noua in lant pentru **fiecare** IP banat (`iptables -A INPUT -s <ip> -j DROP`). Kernel-ul evalueaza lantul secvential, regula cu regula — la cateva mii de IP-uri banate simultan (scenariu plauzibil intr-un DDoS aplicativ sustinut), latentă per pachet crește vizibil, pentru ca fiecare pachet nou trebuie comparat, in cel mai rau caz, cu toate regulile. `ipset` rezolva exact aceasta problema: tine IP-urile intr-o structura de date hash (`hash:ip`), cu lookup **O(1)** indiferent daca sunt 10 sau 100.000 de IP-uri, si suporta expirare nativa per element prin flagul `--timeout` — echivalentul functional exact al `flags timeout` din setul nftables folosit in Capitolul 5.6.

Structura actiunii (iptables-drupal-ban.conf)

Actiunea urmeaza aceeasi logica `actionstart / actionban / actionunban / actioncheck` ca varianta nftables, dar cu comenzi specifice ipset/iptables:

Etapă	Ce face
<code>actionstart</code>	Creeaza doua seturi (<code>hash:ip</code> pentru IPv4, <code>hash:ip family inet6</code> pentru IPv6) si insereaza cate o singura regula <code>-m set --match-set <name> src -j DROP</code> in lantul <code>INPUT</code> , o data, la prima pornire
<code>actionban</code>	<code>ipset add <name> <ip> timeout <bantime> -exist</code> — adauga elementul cu expirare automata, fara sa atinga regula iptables
<code>actionunban</code>	<code>ipset del <name> <ip> -exist</code> — pentru un unban manual explicit; expirarea automata prin <code>timeout</code> face acest pas rar necesar in practica
<code>actioncheck</code>	<code>ipset list <name></code> — fail2ban verifica periodic ca setul inca exista (ex: nu a fost sters manual)

Rezultatul functional este identic cu varianta nftables: pachetele de la un IP banat sunt aruncate de kernel, in lantul `INPUT`, inainte sa ajunga la nivelul TCP/HTTP al nginx-ului — nginx nu "vede" niciodata cererea.

Protectia de baza L3/L4 (00-ddos-base-iptables.sh)

Scriptul reproduce, cu module iptables echivalente, aceeasi protectie de retea din `00-ddos-base.nft`:

Protectie	Modul nftables (Cap. 5.6 / arhiva)	Modul iptables echivalent
SYN flood	<code>limit rate 60/second burst 20</code>	lant dedicat <code>SYN_FLOOD</code> cu <code>-m limit --limit 60/second --limit-burst 20</code>

Semnatura SYN scan (MSS mic)	<code>tcp option maxseg size set 1-536 drop</code>	<code>-m tcpmss --mss 1:536 -j DROP</code>
ICMP flood	<code>icmp type echo-request limit rate 10/second</code>	<code>-m limit --limit 10/second</code>
Conexiuni concurente/IP	<code>meter conn-per-ip { limit rate over 100/second }</code>	<code>-m connlimit --connlimit-above 100 --connlimit-mask 32</code>

`connlimit` masoara conexiuni **concurente** (deschise simultan), nu rata de conexiuni noi pe secunda ca meter-ul `nftables` — cea mai apropiata aproximare disponibila nativ in `iptables` fara module suplimentare (ex: `xt_recent`). Pentru majoritatea site-urilor Drupal, pragul de 100 de conexiuni concurente per IP e suficient de permisiv pentru trafic legitim si suficient de strict pentru a limita un flood aplicativ.

Persistenta la reboot

Spre deosebire de `nftables` (unde `fail2ban` recreaza tabela la fiecare pornire prin `actionstart`), regulile de baza `iptables` NU se salveaza automat. Dupa rularea `00-ddos-base-iptables.sh`, rulati `netfilter-persistent save` (pachetul `iptables-persistent`) pentru ca regulile sa supravietuiasca unui restart de server. Seturile `ipset` create de actiunea `fail2ban` se recreaza automat la fiecare pornire a serviciului `fail2ban`, la fel ca la `nftables`.

5.9 Whitelisting — evitarea auto-banarii

Inainte de a activa pipeline-ul in productie, adaugati explicit IP-ul/IP-urile voastre de administrare in `ignoreip` din `[DEFAULT]` (fisierul `drupal-nginx.local`). `Fail2ban` verifica aceasta lista **inaintea** oricarei evaluari de filtru — un IP din `ignoreip` nu va fi niciodata banat, indiferent de scor.

6. Testare si validare

6.1 Validarea sintaxei si a regex-urilor

shell

```
nginx -t
fail2ban-client -t

# testeaza un filtru direct pe un fisier de log existent
fail2ban-regex /var/log/nginx/security-alerts.log \
/etc/fail2ban/filter.d/drupal-security-events.conf
```

6.2 Simulare controlata (DOAR pe staging)

shell

```
./scripts/test_attack_simulation.sh https://staging.exemplu.md

tail -f /var/log/nginx/security-alerts.log
fail2ban-client status drupal-security-events

# varianta nftables:
nft list set inet f2b-drupal drupal-critical

# varianta iptables+ipset:
ipset list drupal-critical
```

Rulati scriptul de simulare exclusiv pe infrastructura proprie de staging. Generarea de trafic abuziv catre sisteme care nu va apartin poate constitui o infractiune, indiferent de intentie.

6.3 Verificare end-to-end

Dupa simulare, IP-ul de test ar trebui sa apara in trei locuri, in aceasta ordine cronologica: (1) `security-events.log` — evenimentele brute; (2) `security-alerts.log` — linia `SECURITY_ALERT` agregata; (3) output-ul `nft list set` — elementul cu timeout-ul ramas afisat.

7. Monitorizare si mentenanta

Comanda	Scop
<code>fail2ban-client status</code>	Lista jail-urilor active si numarul de IP-uri banate curent
<code>fail2ban-client status drupal-security-events</code>	Detaliu: IP-uri banate in acest jail specific
<code>nft list table inet f2b-drupal</code>	[nftables] Starea completa a seturilor/regulilor de banare active
<code>ipset list drupal-critical</code>	[iptables] Echivalentul de mai sus pentru varianta ipset
<code>systemctl status drupal-security-processor</code>	Verifica ca procesorul Python ruleaza fara erori
<code>journalctl -u drupal-security-processor -f</code>	Urmarire live a alertelor generate (logging catre stdout/journal)

Recomandare de mentenanta: configurati `logrotate` pentru `security-events.log` si `security-alerts.log` (implementarea "tail -F" a procesorului deja gestioneaza rotatia fara pierdere de date), si revizuiti trimestrial pragurile de scor din `SCORE_RULES` in functie de tiparele reale de trafic observate.

8. Troubleshooting

Simptom	Cauza probabila	Solutie
Utilizatori legitimi primesc 429	Rate limit prea strict, sau real IP nu e configurat in spatele unui CDN	Verificati <code>01-realip.conf</code> ; ajustati <code>rate / burst</code> in zona relevanta
Niciun IP nu e banat desi atacul continua	Serviciul procesorului nu ruleaza, sau <code>ignoreip</code> include din greseala IP-ul atacatorului	<code>systemctl status drupal-security-processor</code> ; verificati <code>jail.d/drupal-nginx.local</code>
<code>nft: Error: Could not process rule la actionban</code>	Tabela <code>f2b-drupal</code> nu a fost creata (actionstart a esuat)	<code>fail2ban-client reload</code> ; verificati manual <code>nft add table inet f2b-drupal</code>
Procesorul nu vede linii noi dupa logrotate	Permisuni incorecte pe fisierul nou creat de logrotate	Asigurati <code>create 0640 www-data adm</code> in configuratia logrotate
V-ati auto-banat propriul IP	IP-ul de admin lipseste din <code>ignoreip</code>	<code>fail2ban-client set <jail> unbanip <ip></code> , apoi adaugati IP-ul in <code>ignoreip</code>

9. Anexa: lista completa de fisiere din arhiva

Fisier	Destinatie pe server
nginx/conf.d/00-security-globals.conf	/etc/nginx/conf.d/
nginx/conf.d/01-realip.conf	/etc/nginx/conf.d/
nginx/sites-available/drupal-site.conf	/etc/nginx/sites-available/
fail2ban/filter.d/nginx-limit-req.conf	/etc/fail2ban/filter.d/
fail2ban/filter.d/nginx-botsearch.conf	/etc/fail2ban/filter.d/
fail2ban/filter.d/drupal-security-events.conf	/etc/fail2ban/filter.d/
fail2ban/jail.d/drupal-nginx.local	/etc/fail2ban/jail.d/ [nftables]
fail2ban/action.d/nftables-drupal-ban.conf	/etc/fail2ban/action.d/ [nftables]
fail2ban/jail.d/drupal-nginx-iptables.local	/etc/fail2ban/jail.d/drupal-nginx.local [iptables]
fail2ban/action.d/iptables-drupal-ban.conf	/etc/fail2ban/action.d/ [iptables]
iptables/00-ddos-base-iptables.sh	rulare directa + netfilter-persistent save [iptables]
scripts/security_log_processor.py	/opt/drupal-antiddos/scripts/
scripts/test_attack_simulation.sh	(rulare locala, doar pe staging)
systemd/drupal-security-processor.service	/etc/systemd/system/
nftables/00-ddos-base.nft	/etc/nftables.d/ [nftables]